

Investments: Machine Learning for Finance

Áron Miklós, Konrad Ochedzan, Monna Dimitrova[◇]

Abstract—This study evaluates six neural network architectures for S&P 500 return prediction across expanding and rolling window training strategies, with and without autoencoder preprocessing. LSTM with autoencoder preprocessing emerged as the optimal architecture. It achieved the lowest MSE in both training strategies and demonstrated superior stability across different market conditions. Autoencoder preprocessing provided substantial benefits for simpler architectures like feedforward networks, but showed mixed effects on more sophisticated models. Diebold-Mariano tests confirmed statistically significant differences between model predictions, with p-values approaching zero for most pairwise comparisons. Different architectures captured complementary aspects of market behavior—LSTM with autoencoder provided stable predictions while TFT without autoencoder showed higher sensitivity to more extreme market events.

I. INTRODUCTION

Over the past decade, machine learning has become increasingly important in finance, changing how investment firms make decisions and manage portfolios. Traditional portfolio management has long relied on fundamental analysis, technical charts, and the experience of human traders. While these approaches remain valuable, they struggle to keep pace with today's data-rich financial markets.

Modern financial markets generate massive amounts of data every second — price movements, trading volumes, news sentiment, economic indicators, and more. This creates both an opportunity and a challenge. The opportunity lies in the potential insights hidden within this data; the challenge is that no human analyst can process it all effectively. This is where machine learning becomes particularly useful.

Machine learning transforms portfolio management in four ways. Algorithms identify recurring market behaviors across different timeframes, from minutes to months. ML models analyze asset correlations and volatility patterns, helping managers understand and control portfolio risk more effectively. This leads to smarter position sizing and diversification decisions. By combining price data, economic indicators, and market sentiment, machine learning creates trading signals that process far more information than traditional methods allow. Unlike rigid trading rules, ML models learn continuously from new data and adjust to changing market conditions, keeping strategies relevant as markets evolve.

II. OBJECTIVE

This project aims to build and test a machine learning system that can predict S&P 500 returns. We chose the S&P 500 because it represents the broader U.S. stock market, has plenty of historical data to work with, and is actively traded, making our results potentially practically relevant.

A. Why Trading Depends on Predictions

Every trading decision fundamentally is a forecast about future market conditions. When investors purchase securities, they implicitly predict price appreciation; on the other hand, selling decisions reflect expectations of decline or relative underperformance. Even passive buy-and-hold strategies embody predictions about long-term market growth trajectories.

Trading decisions need predictions across multiple dimensions:

Price Direction: The fundamental question concerns whether asset prices will appreciate or depreciate.

Price Magnitude: Beyond directional predictions, traders must estimate the expected magnitude of price movements. Small anticipated returns may not justify transaction costs and risk exposure.

Temporal Dynamics: The timing of predicted price movements affects strategy performance as poor timing can generate substantial losses.

Prediction Confidence: Quantifying forecast uncertainty helps for an optimal position sizing and risk management. Higher confidence predictions may justify larger allocations, while uncertain forecasts warrant more conservative positioning.

To make these predictions, traditional approaches often rely on either fundamental or technical analysis. Those methods have limitations when dealing with the complexity and speed of modern markets.

Our machine learning approach tries to improve on traditional methods in the several ways previously mentioned.

III. NETWORK ARCHITECTURES

Neural networks come in various types, each suited to different financial data challenges. Stock prices are noisy, relationships shift over time, and patterns can quickly become obsolete. Some networks capture temporal trends well, while others uncover hidden structures or manage high-dimensional data. We tested several architectures to find the most effective for our trading algorithm.

[◇] Master of Science UZH ETH in Quantitative Finance

A. Deep Feed-Forward Networks (Dense Layers)

Feed-forward networks use a simple architecture where information flows one way from input to output through fully connected hidden layers. Each layer performs a linear transformation followed by a non-linear activation:

$$h_l = \sigma(W_l h_{l-1} + b_l) \quad (1)$$

$$\hat{y} = W_{out} h_L + b_{out} \quad (2)$$

where σ is the activation function that enables the network to capture non-linear relationships in financial data.

In theory, a deep enough dense network can approximate any function. They're also computationally efficient and learn complex feature interactions automatically.

In our setup, feedforward networks include a basic autoregressive element by using past returns as input features. This adds simple temporal context, capturing some historical market data without the complexity of LSTM memory mechanisms.

However, feedforward networks have limitations with financial time series. They make predictions independently, relying only on provided features with no memory of past patterns. They require a fixed input size, which limits flexibility with varying data availability. Additionally, they lack time awareness, treating all periods equally regardless of market cycles or seasonal effects.

This is why we use them mainly as components within more sophisticated architectures.

B. Autoencoders

Autoencoders are a type of neural network that learns to compress data and then reconstruct it.

The network has two main parts:

Encoder: Compresses input data into a smaller representation

Decoder: Reconstructs the original data from the compressed version

The training goal to minimize the difference between input and output:

$$L = \|x - \text{decoder}(\text{encoder}(x))\|^2 \quad (3)$$

We use encoders since they are very useful for dimensionality reduction of correlated indicators, feature extraction that captures market essence, anomaly detection during unusual market conditions, and noise reduction in messy financial data.

C. Recurrent Neural Networks

Recurrent Neural Networks (RNNs) maintain memory through hidden states that propagate temporal dependencies:

$$h_t = \tanh(W_{hh} h_{t-1} + W_{hx} x_t + b_h) \quad (4)$$

$$y_t = W_{hy} h_t + b_y \quad (5)$$

This architecture enables the network to incorporate historical context when making predictions, which is important for capturing momentum and trend patterns in financial markets. In our paper, we use the LSTM model.

1) *Long Short-Term Memory (LSTM)*: Basic RNNs suffer from memory degradation over long sequences. They lose earlier information through vanishing gradients. LSTMs address this limitation through gated memory mechanisms that selectively preserve relevant information from the past.

LSTMs employ three gates to control information flow: forget gates determine what to discard from previous states, input gates select new information to store, and output gates control what information to release:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (6)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (7)$$

$$C_t = f_t \times C_{t-1} + i_t \times \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (8)$$

$$h_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \times \tanh(C_t) \quad (9)$$

2) *Attention Mechanisms*: LSTMs compress history into fixed-size states, which can lose important context. Attention mechanisms solve this by letting models selectively focus on any past time step, highlighting which historical events are most relevant for current predictions.

D. Convolutional Neural Networks

Convolutional Neural Networks (CNNs) were originally developed for image processing but can be adapted for time series analysis by treating sequential data as 1D signals. They involve sliding filters (kernels) across the input sequence to detect local patterns.

The fundamental CNN operation for time series is the 1D convolution:

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m] \cdot g[n - m] \quad (10)$$

In practice, for a finite sequence, this becomes:

$$y[n] = \sum_{k=0}^{K-1} w[k] \cdot x[n - k] + b \quad (11)$$

where $w[k]$ represents the learnable filter weights, K is the kernel size, and b is the bias term.

CNNs offer key advantages for financial time series: they learn temporal patterns automatically, use multiple filters to detect diverse market behaviors, and pooling provides translation invariance, recognizing patterns regardless of timing. Their hierarchical design lets lower layers capture short-term moves and deeper layers spot longer-term trends.

E. Temporal Fusion Transformer (TFT)

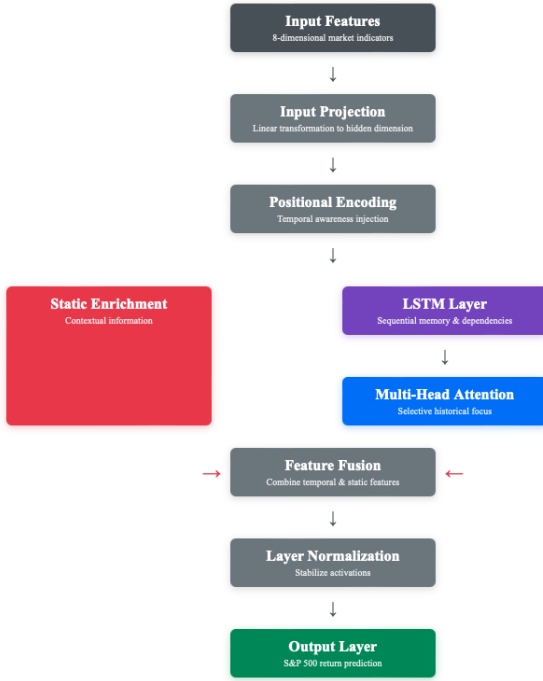


Fig. 1. Temporal Fusion Transformer (TFT) with LSTM Layer Architecture

This is a hybrid architecture combining LSTM sequential processing with transformer attention mechanisms. It creates a more comprehensive framework for temporal pattern recognition.

The TFT has 4 components: LSTM layers for sequential memory retention, multi-head attention for selective historical focus, positional encoding for temporal awareness, and static enrichment networks for contextual information integration. The idea is to capture both local temporal dependencies and long-range market patterns.

F. Regularization Techniques

Training neural networks on financial data requires careful regularization due to market non-stationarity and the risk of overfitting to historical patterns that may not persist.

1) *ElasticNet Regularization*: We implement custom loss functions incorporating both L1 and L2 penalties to constrain model complexity and improve generalization to unseen market conditions.

ElasticNet combines two types of penalties:

$$R(\theta) = \alpha [l_{ratio} \cdot \|\theta\|_1 + (1 - l_{ratio}) \cdot \|\theta\|_2^2] \quad (12)$$

- **L1 penalty** (the first term): Encourages sparsity—pushes some weights to exactly zero, effectively doing feature selection
- **L2 penalty** (the second term): Keeps weights small and stable, preventing any single feature from having too much influence

2) *Other Regularization Techniques*: We also use several complementary regularization techniques:

Gradient Clipping: We implement gradient clipping to prevent exploding gradients, particularly important for recurrent architectures like LSTMs and attention-based models during volatile market periods:

$$\hat{g} = \begin{cases} g & \text{if } \|g\| \leq \tau \\ \frac{\tau \cdot g}{\|g\|} & \text{if } \|g\| > \tau \end{cases} \quad (13)$$

where τ is the clipping threshold, typically set to 1.0 in our experiments.

Dropout: Randomly deactivates neurons during training to prevent over-reliance on specific features and improve generalization to unseen market conditions. We apply dropout rate of 10%.

Group Normalization: Stabilizes training by normalizing layer activations, which is particularly beneficial for financial time series where input magnitudes can vary significantly across different market regimes.

Weight Initialization: Proper initialization prevents gradient issues and ensures stable training convergence. Our initialization strategy varies by architecture:

- **Dense/Linear layers**: Xavier uniform initialization for layers with sigmoid/tanh activations.
- **ReLU-activated layers**: Kaiming normal initialization to maintain activation variance.
- **LSTM gates**: Orthogonal initialization for recurrent weights to prevent vanishing/exploding gradients in temporal dependencies.
- **Attention mechanisms**: Scaled initialization based on attention dimension to ensure stable attention weight computation.

Bias Initialization: All biases are initialized to zero, except for LSTM forget gates which are initialized to 1.0 to encourage information retention in early training stages.

IV. EXPERIMENTS

A. Choice of Signals

Feature selection is an important component of financial machine learning - it requires indicators that capture diverse market dynamics while still minimizing redundancy and noise.

1) *Signal Transformation Through Autoencoders*: Testing models with and without autoencoder preprocessing evaluates two distinct signal sets: raw features as the original multi-dimensional space, and autoencoder outputs as compressed, denoised representations. This helps determine if learned feature transformations enhance predictive power over using technical and macroeconomic indicators directly.

2) *Dataset Construction*: The S&P 500 data that we use incorporates several feature categories:

Technical Indicators: Price and volume-derived features including moving averages, momentum indicators (RSI, MACD), volatility measures, and volume-based metrics.

Macroeconomic Variables: Our feature set incorporates several economic indicators including 3-month Treasury bill

rates, term spreads between long and short-term rates, exchange rate indicators, volatility indices, and credit spreads.

Our final feature set has 22 dimensions.

B. Choice of Network Type and Parameters

Our experiments compare six neural network types across multiple parameter configurations to identify optimal architectures for financial forecasting. We used the following three-phase experimental process.

Phase 1: Architecture Selection First, we test different model architectures to see which general approach works best for each model type. We analyzed following configurations for each model type:

Temporal Fusion Transformers (TFT): We tested hidden dimensions (64 vs. 128), attention heads (4 vs. 8), network depth (2 vs. 3 transformer layers). Additional parameters included LSTM hidden sizes matching the transformer dimensions and positional encoding variants.

Long Short-Term Memory (LSTM): We explored different architectural configurations including hidden layer sizes (64, 200, 256), number of LSTM layers (2, 3).

FeedForward Networks: We tested various layer widths ([100, 50, 25], [200, 100, 50], [300, 150, 75]). Activation functions included ReLU, Leaky ReLU, and GELU, with dropout applied between layers.

Convolutional Neural Networks (CNN): We evaluated multiple filter configurations with varying numbers of convolutional layers (3, 4), and different channel progressions ([32, 64, 32], [32, 64, 64, 32], [48, 72, 72, 48]). Pooling strategies included max pooling, average pooling, and adaptive pooling.

Standard Transformers: We tested encoder-only architectures using model dimensions of 128 and 256, 8 attention heads, and 2 or 3 encoder layers. The grid included a fixed dropout rate of 0.1 and a maximum sequence length of 500.

Each architecture was evaluated both with and without autoencoder preprocessing, creating 12 distinct model-preprocessing combinations for each training procedure.

Phase 2: Hyperparameter Tuning Next, we fine-tune training of the promising architectures optimizing for number of epochs, batch size, regularization parameters, sequence length and learning rate.

Phase 3: Final Model Training Finally, we train models using hyper-parameters and architectures that achieved the lowest loss.

1) Evaluation Methodology: Time Series Cross-Validation Structure: Our evaluation uses temporal splits, with each fold representing a distinct time period. Unlike random cross-validation, we preserve chronological order—training on past data and testing on future periods—ensuring realistic, out-of-sample evaluation that reflects actual trading conditions.

Rolling Window Training: In the rolling window phase, we maintain a fixed training window size that “rolls” forward through time:

- Training window: Fixed size (e.g. year is from March 1st to March 1st)
- At each step, the oldest data is dropped and newest data is added
- Testing: Next period after training window
- Advantage: Focuses on recent market patterns, adapts quickly to regime changes
- Challenge: Limited historical context, may miss long-term patterns

Expanding Window Training: In the expanding window approach, the training set grows continuously:

- Training window: Starts small, incorporates all historical data up to test point
- At each step, only new data is added (nothing is dropped)
- Testing: Same next-period approach as rolling windows
- Advantage: Maximum historical information, stable parameter estimates
- Challenge: May include outdated patterns, slower adaptation to new regimes

2) Training Protocol: We maintained a standardized training framework across all models for fair comparison, with minor architecture-specific optimizations (such as AdamW optimizer for Transformer-based models and adjusted learning rate schedules) applied only when theoretically justified. As a training objective we decided to use mean-squared error (MSE). We also experimented with including Sharpe ratio in the loss function. This approach, however, didn’t yield satisfactory results.

Following a architecture selection process, we proceeded with training and deployment of the optimal model configurations for a comparative analysis. The training process employed time series cross-validation across multiple folds, with models generating out-of-sample predictions spanning from March 2012 to February 2025.

3) Research Hypotheses: Hypothesis 1 (Autoencoder Benefits): We expect autoencoders to improve performance for simpler architectures—FeedForward networks and CNNs—that do not have built-in feature learning mechanisms. These architectures should benefit from the dimensionality reduction and noise filtering provided by autoencoder preprocessing.

Hypothesis 2 (Autoencoder Limitations): We hypothesize that autoencoders will provide minimal or negative impact for advanced architectures—LSTMs, Transformers, and TFTs—which already possess strong feature learning capabilities through recurrent connections or attention mechanisms. For these models, autoencoder compression may remove useful information.

Hypothesis 3 (Window Strategy Effects): We hypothesize that expanding window training will benefit complex, flexible architectures (TFTs, Transformers, LSTMs) by providing more historical data to learn from. On the other hand, simpler architectures (FeedForward, CNNs) will perform better with rolling windows.

C. Results

Table I presents the performance comparison of all architectures ranked by average MSE across all validation folds.

TABLE I
OUT-OF SAMPLE MSE PERFORMANCE COMPARISON ACROSS TRAINING STRATEGIES

Architecture	Rolling	Expanding	Difference
LSTM + Autoencoder	0.092	0.090	+0.002
LSTM	0.226	0.126	+0.100
Transformer + Autoencoder	0.106	0.100	+0.006
Transformer	0.569	0.111	+0.458
TFT + Autoencoder	0.111	0.115	-0.004
TFT	0.131	0.223	-0.092
FeedForward + Autoencoder	0.160	0.123	+0.037
FeedForward	5.900	0.323	+5.577
CNN + Autoencoder	0.216	0.158	+0.058
CNN	1.521	0.143	+1.378

Note: MSE values are multiplied by 10^{-3} . Positive differences indicate higher MSE in rolling windows.

1) Specific Model Results:

a) *LSTM*: Based on the MSE criterion, LSTM with autoencoder outperforms its peers across both strategies. However, as shown in Figure 2, its predictions follow a stable pattern but lack the flexibility to capture return volatility. Instead, the model appears to regress toward the long-term mean.

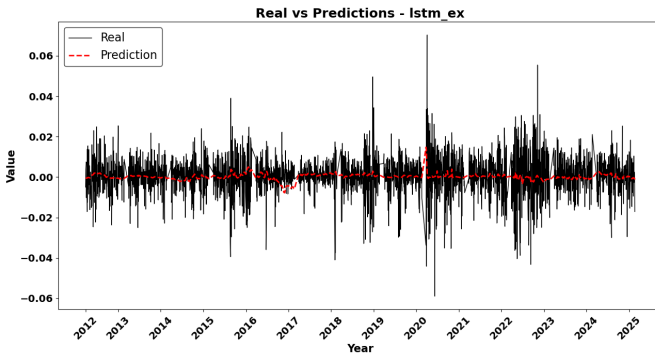


Fig. 2. LSTM predictions, with autoencoder, expanding procedure

b) *Transformers*: Transformers with autoencoder show similar behavior, as shown in Figure 3. The difference is that transformers appear to capture the short-term mean for each fold.

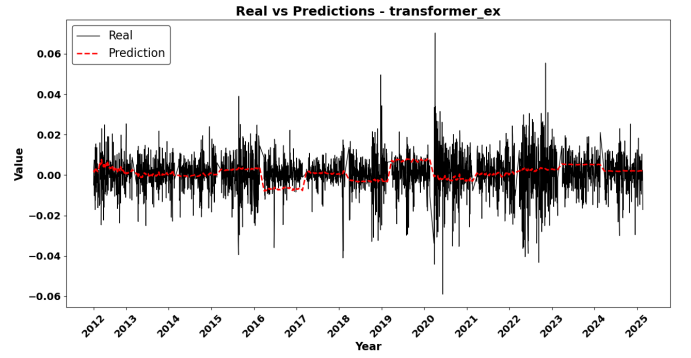


Fig. 3. Transformer predictions, with autoencoder, expanding procedure

Moreover, standard Transformers exhibited high sensitivity to training window choice, with the non-autoencoder version dropping from rank 3 in expanding windows to rank 8 in rolling windows (MSE = 0.569). This indicates that Transformer architectures require substantial historical data to perform optimally without auto-encoder.

c) *Temporal Fusion Transformers*: TFTs were the only model to perform better with rolling windows than with expanding ones, indicating their attention mechanisms are more responsive to recent data patterns than to long-term history.

While the autoencoder version had lower MSE, Figures 4 and 5 show that the non-autoencoder model better captures market volatility rather than regressing to long-term means, which is advantageous for financial forecasting.

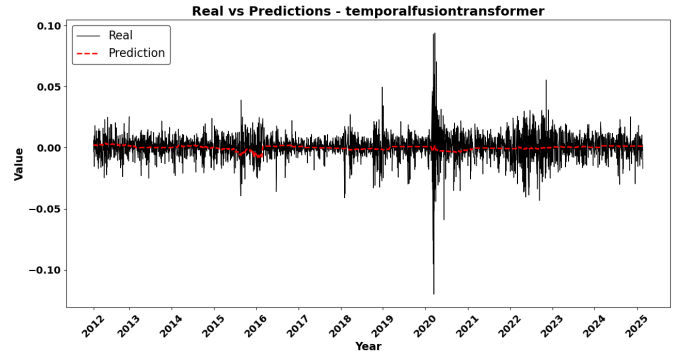


Fig. 4. TFT predictions, with autoencoder, rolling procedure

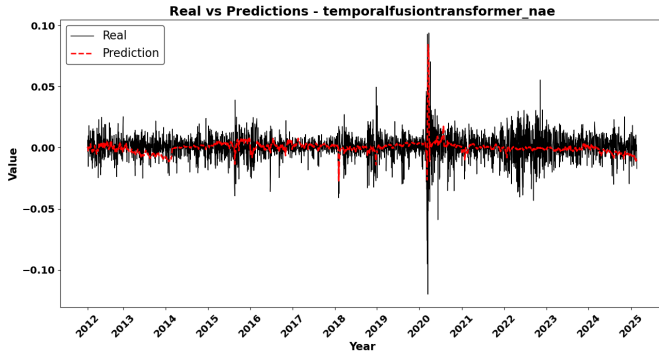


Fig. 5. TFT predictions, without autoencoder, rolling procedure

d) *Feed Forward Neural Networks*: For Feed Forward networks, adding an autoencoder improves performance under both training regimes. Additionally, the expanding window helps regularize loss for the model without an autoencoder, resulting in notable performance gains. Figures 7 and 6 illustrate how the autoencoder effectively stabilizes predictions for that model.

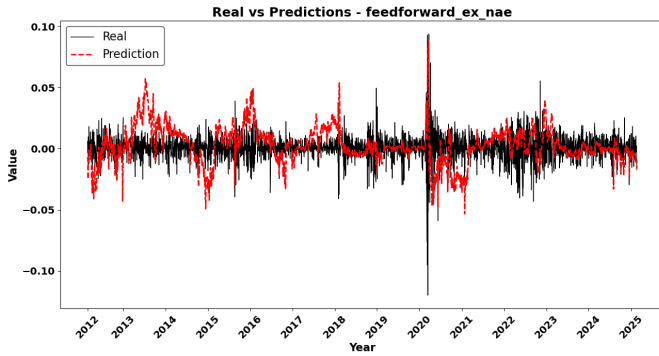


Fig. 6. FeedForward predictions, without autoencoder, expanding procedure

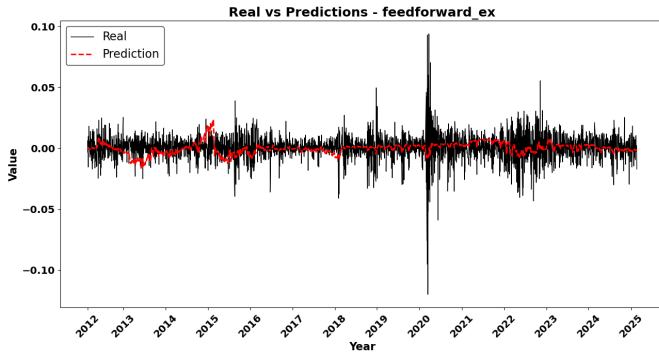


Fig. 7. FeedForward predictions, with autoencoder, expanding procedure

e) *Convolutional Neural Networks*: CNNs show similar behavior to Feed Forward networks. The expanding procedure lowers average MSE, especially for models without autoencoders. With the rolling procedure, the autoencoder model

performs better, but under the expanding setup, the non-autoencoder model outperforms. This suggests that access to more training data may help CNNs capture more complex structures.

2) *Autoencoder Impact Analysis*: The addition of autoencoder preprocessing showed mixed results across architectures, with significant improvements for some models and degradation for others:

Positive Impact Architectures (Rolling Window):

- **FeedForward**: +97.3% MSE improvement (5.900 → 0.160)
- **CNN**: +85.8% MSE improvement (1.521 → 0.216)
- **Transformer**: +81.4% MSE improvement (0.569 → 0.106)
- **LSTM**: +59.2% MSE improvement (0.226 → 0.092)
- **TFT**: +15.26% MSE improvement (0.131 → 0.111)

Positive Impact Architectures (Expanding Window):

- **FeedForward**: +61.9% MSE improvement (0.323 → 0.123)
- **TFT**: +48.4% MSE improvement (0.223 → 0.115)
- **LSTM**: +28.6% MSE improvement (0.126 → 0.090)
- **Transformer**: +9.9% MSE improvement (0.111 → 0.100)

Negative Impact Architectures:

- **CNN (Expanding)**: -10.5% MSE degradation (0.143 → 0.158)

3) *Sharpe Ratio*: We calculated Sharpe ratios by taking long positions on positive predictions and short positions on negative ones, then comparing these to actual returns to derive strategy returns. On average, out-of-sample Sharpe ratios were negative across all models, while in-sample ratios were positive only for Feed Forward and TFT models without autoencoders. However, many folds did show positive Sharpe ratios both in- and out-of-sample. The negative averages are driven by a few periods with large losses, reflected in high standard deviations across folds, typically ranging from 10 to 20.

4) *Hit Rate*: Hit rate (HR) was calculated as the percentage of correctly predicted signs of returns.

TABLE II
AVERAGE HIT RATE (%) ON TRAIN AND TEST SETS, COMPARING EXPANDING AND ROLLING WINDOW PROCEDURES

Architecture	Train (E)	Test (E)	Train (R)	Test (R)
LSTM	56.9	52.9	50.5	49.1
LSTM + AE	54.3	52.7	60.3	52.0
FeedForward	62.9	54.2	59.8	51.9
FeedForward + AE	54.6	53.6	58.6	55.2
TFT	59.2	58.4	51.8	53.5
TFT + AE	56.6	57.0	55.2	49.6
Transformer	51.4	49.7	49.6	50.0
Transformer + AE	52.5	50.7	54.4	50.6
CNN	51.6	49.7	52.4	50.8
CNN + AE	48.7	47.6	51.3	51.6

Note: All hit rate values are expressed as percentages. "E" and "R" refer to expanding and rolling window training procedures, respectively. "AE" refers to autoencoder.

We can see that for almost all models we managed to achieve hit rates above 50% out-of-sample, with TFT without autoencoder trained on expanding windows showing the highest accuracy.

5) *Findings and Implications:*

- 1) **Hypothesis 1 Confirmed - Autoencoder Benefits for Simple Architectures:** Our results support our first hypothesis that autoencoders improve performance for simpler architectures lacking built-in feature learning mechanisms. FeedForward networks showed improvement (+61.9% MSE reduction: $0.323 \rightarrow 0.123$ in expanding window, +97.3% in rolling window). This validated our expectation that simple models require external feature learning support. CNNs showed mixed results, with slight performance degradation in expanding windows ($0.143 \rightarrow 0.158$) but improvement in rolling windows (+85.8%), suggesting architecture-specific responses to different training strategies.
- 2) **Hypothesis 2 Partially Confirmed - Trade-offs with Advanced Architectures:** While advanced architectures did benefit from autoencoders in terms of MSE reduction - with LSTM achieving top performance (ranks 1 in both strategies) and TFT improving significantly in expanding windows (rank 4 vs 9) - this improvement comes at the cost of prediction flexibility. As evidenced in the model-specific results, autoencoder preprocessing tends to flatten predictions for complex architectures, causing models to fit closer to the mean rather than capturing market volatility and trends. For instance, LSTM with autoencoder produces stable, near-mean predictions despite achieving the lowest MSE, while models without autoencoders show greater variability and better capture extreme market movements. This suggests that for advanced architectures, autoencoders may remove useful information about market dynamics even while improving traditional accuracy metrics, supporting our original hypothesis about the potential drawbacks of preprocessing for sophisticated models.
- 3) **Hypothesis 3 Not Supported - Window Strategy Effects:** Our third hypothesis regarding window strategy effects was not supported by the results. Rather than complex architectures benefiting from expanding windows and simple ones from rolling windows, we observed biggest improvements in average MSE for Feed Forward and Convolutional Neural Networks. Moreover, the by far most complex architecture, that is TFT, performed worse with expanding strategy. What's more, relative performance differences between architectures were inconsistent across window strategies - for example, TFT's ranking varied a lot between strategies (rank 4-9 in expanding vs rank 3-4 in rolling). This indicates that the relationship between architectural complexity and optimal training strategy is more nuanced than anticipated.
- 4) **Complementary Model Behaviors:** Beyond our initial hypotheses, we discovered that different architec-

tures capture complementary aspects of market behavior. While LSTM with autoencoder achieved optimal MSE performance and stability, TFT without autoencoder demonstrated superior hit rates (58.4% in expanding windows) and better captured extreme market events, despite lower overall accuracy. This suggests that model selection should consider not just prediction error but also the specific market dynamics most relevant to the trading application.

6) *Training and Forecasting Results:*

a) *Model Training and Deployment:* Diebold-Mariano tests confirm that the different neural network architectures produce statistically significantly different predictions, with p-values approaching zero for nearly all pairwise comparisons. This validates that architectural choice matters a lot in financial forecasting - the models are not simply producing equivalent predictions with minor variations. The only exception was the comparison between LSTM and TFT (p-value = 0.371), suggesting these two architectures may capture similar market patterns.

b) *Temporal Pattern Analysis:* Temporal dynamics across models reveal critical differences in adaptability to market conditions. The TFT model without an autoencoder displayed strong temporal adaptation, particularly in its response to different market regimes. Annual breakdowns highlighted its heightened sensitivity during crisis years, with prediction volatility peaking in 2020 ($\sigma = 0.0139$) compared to more stable periods like 2017 ($\sigma = 0.0014$). This suggests it effectively captured time-varying market stress. In contrast, the LSTM with autoencoder maintained consistent performance across time but showed less sensitivity to regime shifts, reflecting its smoother temporal behavior rather than dynamic responsiveness.

c) *Risk Characteristics:* From a risk sensitivity perspective, forecasting models demonstrated a clear trade-off between accuracy and responsiveness to market extremes. The LSTM with autoencoder achieved the lowest MSE, indicating strong average predictive accuracy. However, the TFT without autoencoder captured the broadest prediction range (0.1136) and identified 1.6% of forecasts as extreme events (absolute value $\geq 1\%$), underscoring its greater reactivity to outliers. Meanwhile, the autoencoder-enhanced TFT produced significantly dampened volatility ($\sigma = 0.0017$ vs 0.0053) and entirely eliminated extreme predictions, which may indicate over-regularization that suppresses valuable signal variability in pursuit of improved average accuracy.

V. CONCLUSION

This study evaluated neural network architectures for S&P 500 return prediction, comparing six models across various configurations, with and without autoencoder preprocessing. The results could offer practical guidance and highlight key insights into modeling financial time series.

LSTM with autoencoder preprocessing achieved the best overall performance across both training strategies, but our

analysis revealed important trade-offs between prediction accuracy and practical utility. While autoencoders significantly improved performance for simpler architectures like feedforward networks, their impact on sophisticated models was more nuanced. For advanced architectures, autoencoders improved traditional error metrics but fundamentally altered prediction behavior, creating flatter outputs that regress toward mean values rather than capturing market volatility and extreme events.

Our initial hypotheses about autoencoder effectiveness received mixed validation. Simple architectures benefited substantially from preprocessing as expected, but advanced models showed complex responses that challenged our assumptions about the relationship between architectural sophistication and feature learning requirements. The hypothesis about window strategy effects was not supported, as performance patterns proved inconsistent across different training approaches.

Most importantly, we discovered that different architectures capture complementary aspects of market behavior. Models optimized for statistical accuracy tend to produce conservative, stable predictions, while architectures that score lower on traditional metrics often demonstrate superior sensitivity to market dynamics, regime changes, and extreme events. This suggests that effective financial forecasting may require moving beyond single-model approaches toward systems that balance multiple objectives and leverage the distinct strengths of different neural network architectures.

A. Future Work

1) *Temporal Mixture Models with Gating*: Our results reveal an intriguing pattern: although architectures yielded similar MSEs, they differed markedly in statistical behavior, as shown by the DM tests. TFT effectively captured extreme market events (predictions > 1%), but with higher volatility, while LSTM with autoencoder was stable yet missed outliers. This indicates the models are learning distinct aspects of market dynamics.

This insight suggests a new research direction: instead of selecting a single "best" model, we could design a hybrid system that switches models based on market conditions—using the stable LSTM in calm periods and the more responsive TFT during volatility. A learned gating mechanism could determine which model to trust at each moment.

We would use learnable weights $g_t^{(i)}$ for each model i , giving us:

$$\hat{y}_t = \sum_{i=1}^N g_t^{(i)} \cdot \hat{y}_t^{(i)}$$

The gates would consider market volatility, model confidence, and recent performance to decide the mixing weights. This could give us the best of both worlds - stability when needed, sensitivity when required.

2) *Enhanced Decision-Making Integration*: While our models show strong predictive ability for S&P 500 returns, the Sharpe ratio analysis reflects only a basic approach to turning predictions into trading strategies. Future work should

explore integrating these neural networks with more advanced frameworks—such as smarter portfolio optimization, dynamic position sizing, and robust risk management—to unlock their full potential in real-world applications.

3) *Balancing Stability and Sensitivity in Model Selection*:

A key consideration for future research is the trade-off between model stability and sensitivity to market dynamics. While flatter, more stable models tend to achieve lower MSE, this may not align with the needs of practical trading. For example, LSTM with autoencoder preprocessing lowers MSE but produces mostly flat predictions, whereas without the autoencoder, LSTM outputs are more variable and better capture periods of high returns.

This suggests that potential mixture models could help address this. By combining stable models for baseline predictions with more sensitive architectures for more extreme market movements, we could develop systems that achieve both low prediction error and practical utility for identifying profitable trading opportunities.

REFERENCES

- [1] J. Beran, *Statistics for Long-Memory Processes*. New York: Chapman and Hall, 1994.
- [2] J. Beran, R. Kulik, S. Ghosh, and Y. Feng, *Long-Memory Processes: Probabilistic Properties and Statistical Methods*. Springer, 2013.
- [3] R. N. Bhattacharya, V. K. Gupta, and E. Waymire, "The hurst effect under trends," *Journal of Applied Probability*, vol. 20, pp. 649–662, 1983.
- [4] L. Giraitis, P. Kokoszka, R. Leipus, and G. Teyssière, "Rescaled variance and related tests for long memory in volatility and levels," *Journal of Econometrics*, vol. 112, pp. 265–294, 2003.
- [5] L. Giraitis, P. Robinson, and D. Surgailis, "A model for long memory conditional heteroskedasticity," *Annals of Applied Probability*, vol. 10, pp. 1002–1024, 2000.
- [6] A. W. Lo, "Long-term memory in stock market prices," *Econometrica*, vol. 59, pp. 1279–1313, 1991.
- [7] I. Lobato and P. M. Robinson, "A nonparametric test for $i(0)$," *Review of Economic Studies*, vol. 68, pp. 475–495, 1998.
- [8] D. M. Mason, "The hurst phenomenon and the rescaled range statistic," *Stochastic Processes and their Applications*, vol. 126, pp. 3790–3807, 2016.
- [9] D. Surgailis, H. L. Koul, and L. Giraitis, *Large Sample Inference for Long Memory Processes*. World Scientific, 2012.
- [10] M. S. Taqqu, V. Teverovsky, and W. Willinger, "Estimators for long-range dependence: An empirical study," *Fractals*, vol. 3, no. 4, pp. 785–798, 1995.